

**A Project report presented to the National University in partial
fulfillment of the requirement for the degree of B.Sc. (Hon's) in
Computer Science and Engineering**

Optical Character Recognizer for Bangla (Bangla-OCR)

Md. Mahedi Azad
mahedi2014@gmail.com



**Department of Computer Science and Engineering (CSE), Institute of Science and
Technology, National University, Bangladesh**

Abstract

Optical character recognition (OCR in short) is an important area in pattern recognition and image processing. Research in this field has been carried out since the beginning of the development of digital computers in a number of universities and research institutes, and many solutions to the problem has been proposed and used in commercial systems. However not much can be found about OCRs for the Bangla characters. This project has been an attempt towards the development of a full phased OCR for Bangla characters.

BanglaOCR is currently the only open source optical character recognition (OCR) software for the Bangla (Bengali) script developed by the Center for Research on Bangla Language Processing (CRBLP). Tesseract, maintained by Google, is considered to be one of the most accurate free open source OCR engines currently available. In this paper, we present a new OCR for the Bangla/Bengali script that combines the recognition power of Tesseract and the Bangla script processing power of BanglaOCR by integrating the Tesseract recognition engine into BanglaOCR. We first present the complete methodology to build the combined OCR, followed by the implementation strategy (in python programming language). In this project, we focus on the training data preparation process, Tesseract integration procedure and the post-processing techniques. The techniques described in this paper can be readily applied to build OCRs for other scripts as well.

Contents

Chapter 1: Introduction	1
Tesseract based Bangla-OCR.....	1
Properties of Bangla Scripts.....	1
Mission.....	2
Chapters summary	3
Chapter 2: Study of Related Works	4
Scanning and image digitization	4
Noise Removal	4
Segmentation	5
Line Segmentation Process	5
Word Segmentation Process	5
Character Segmentation Process	6
Matra line detection process	6
Base line detection process.....	6
Character detection	6
Chapter 3: Methodology	7
Preparing Training Data	7
Processing the Document Image	9
Preparing Tesseract supported image	9
Performing Recognition using Tesseract engine	10
Post-processing the generated text output	10
Chapter4: Working flow	11
Chapter 5: Implementation	14
Required Tools and Programming Language	15
Installation of Tesseract OCR Engine	15
Installation of Leptonica Image Processing Library	15
Training Procedure for Bangla for Tesseract OCR Engine	15
Make Box	17
Training file	18
Unicharset.....	18

Font properties18

Clustering.....18

Renaming19

Combining.....19

Move trainingdata.....19

Chapter 1: Introduction

There have been a number of research efforts since the mid 1980's aimed at building OCR systems for recognizing Bangla characters. Most of these (BOCRA and Apona-Pathak) efforts have focused on the recognition of individual characters, rather than on building a complete OCR system. The Center for Research on Bangla Language Processing (CRBLP) released BanglaOCR – the first open source complete OCR software for Bangla – in 2007. BanglaOCR is a complete OCR framework, and has a recognition rate of up to 98% (in limited domains). The Tesseract OCR engine was one of the top 3 engines in the 1995 UNLV Accuracy Test. The algorithms used in the different stages of the Tesseract OCR engine are specific to the English alphabet, which makes it easier to support scripts that are structurally similar by simply training the character set of the new scripts. There are published guidelines about the procedure to integrate Bangla/Bengali language recognition using Tesseract OCR engine[6]. Detailed knowledge of the Bangla script as well as the character training procedure is also given in Python programming language.

Tesseract based Bangla-OCR

Although Tesseract work on English script but we use the Tesseract library in python programming to make Tesseract as “Tesseract based Bangla-OCR” is an open source OCR software for Bangla script recognition that integrates Tesseract's excellent recognition engine into the rest BanglaOCR. It has a graphical user interface (GUI) that enables the user to load in a text image, recognize the image, and then use the built-in spelling checker, which is capable of generating suggestions for misspelled words, to improve the accuracy. This software can be used as an important tool to build lexical resources for the Bangla language.

We briefly present our methodology, implementation details, discuss results, and then finally conclude in the document.

Properties of Bangla Scripts

Main mission and objective of “Tesseract based Bangla-OCR” is to do the recognition rate of BanglaOCR up to 100%, this rate is very sensitive to the typeface and the font size, as well as the document type independent. These are grouped in table-1.1.

Vowels	অ আ ই ঈ উ ঊ ঋ ঌ ঐ ঔ
Consonants	ক খ গ ঘ ঙ চ ছ জ ঝ ঞ ট ঠ ড ঢ ণ ত থ দ ধ ন প ফ ব ভ ম য র ল শ ষ স হ ঙ
Vowel Modifiers	া ি িী ু ূ ে ৈ ো ৌ
Consonant Modifier	্ য ্র
Numerals	০ ১ ২ ৩ ৪ ৫ ৬ ৭ ৮ ৯
Example of Some Compound Characters	ঈ ঙ ঙ ঙ

Figure 1.1: Table-1: Different Types of Bangla Character Sets

Many characters of Bangla script have a horizontal line at upper part called '**matra**' or headline. Bangla text can be partitioned into three zones. These are upper, middle and bottom zones. Upper zone denotes the portion above the matra or headline. Middle zone holds most of the basic and compound characters between the matra and lower base line. Base line is an imaginary line at the bottom end of basic characters and from where bottom zone start where some modifiers are reside. Upper zone holds upper portion of some modifiers and basic characters.

Some common properties in Bangla language are given below:

- Writing style of Bangla is from left to right
- There are no upper and lower case in Bangla language
- Most of the word, the vowels takes modified shape called modifiers or allograph.
- The vowel (A) following a consonant character is never printed in a word. Thus, A (A) can occur only at the beginning of the word.
- There are approximately 253 compound characters.
- There are 32 characters having **matra** or headline with them. The head line is an important feature to locate the script line, to segment the characters in a word and to classify the characters using binary tree classifier. Some characters have a signature extended above the head line (e.g. B, C, D), which is also useful for character classification. **Matra** is also useful for skew detection. (BB)
- Some characters have left side bar and some have right side bar.
- Many characters have one or more corner or sharp angle property. Some characters carry isolated dot along with them.

Mission

Main mission and objective of “Tesseract based Bangla-OCR” is to do the recognition rate of BanglaOCR up to 100%, this rate is very sensitive to the typeface and the font size, as well as the document type independent.

Chapters summary

Chapter 2 presents related works on OCR and a brief of major research work on Bangla OCR. In chapter 3 we discuss the overall methodology, chapter 4 working flow and techniques used for preprocessing. Features and their extraction procedures have been presented at chapter 3. Classification, training and final recognition have been presented at chapter 5 as the part of implementation.

Chapter 2: Study of Related Works

This chapter presents related works on OCR and a brief of major research work on Bangla OCR. Optical character recognition is performed in three major steps: preprocessing, feature extraction and recognition. To make a scanned document for feature extraction and for further handing out, some preprocessing sub-steps should be performed.

Scanning and image digitization

For scanning most of the author suggest for using automatic flat-bed scanner rather handheld scanner because of uniform scanning [1][11]. Then the original image can be converted into a gray-image. Grey level of each pixel is determined by the average of red, green and blue levels of that pixel [12]. Some authors suggest to binarization of gray scale image binarization separates the foreground (text) and background information [1].

Noise Removal

Noise can be produced during the scanning section automatically. Two types of noises are common. They are background noise and salt & paper noise. Complex script like Bangla, we cannot eliminate wide pixels from upper or lower portion of a character because it may not only eliminate noise but also the difference between two characters like "ই" and "ঐ" and for some other characters [13]. Some noise detection methods are given below:

- The commonly used approach is, to low pass filter the image and to use it for later processing. The objective in the design of a filter to reduce noise is that it should remove as much of the noise as possible while retaining the entire signal [1].
- Mixed noise of Gaussian and impulse cannot be removed by conventional method at the same time. In [14], the author proposed an enhanced TV (Total Variation) filter which can remove these two types of noise based on PSNR (Pick Signal to Noise Ratio) and subjective image quality. The main advantage of this method is it can eliminate mixed noise efficiently & quickly.
- Comparing with the conventional method such as averaging method and the median method, the proposed method in [15] gets higher quality. This method is based on kalman filter.

- Dots existing in a character like *i* may be treated as noise. In [16], the author proposed a new method for removing noises similar to dots from printed documents. In this method, first estimate the size of the dots in each region of the text. Then the minimum size of dots in each region is estimated based on the estimated size of dot in that region.
- In [17], the authors used connected component information and eliminated the noise using statistical analysis for background noise removal. For other type of noise removal and smoothing they used wiener and median filters [18]. Connected component information is found using boundary finding method (such as edge detection technique). Pixels are sampled only where the boundary probability is high. This method requires elaboration in the case where the characteristics change along the boundary.
- In [4], noise is removed from character ঙ images. Noise removal includes removal of single pixel component and removal of stair case effect after scaling. However, this does not consider background noise and salt and paper noise.

Segmentation

Before applying feature's extraction procedure, most of the authors suggest segmenting the scanned text into following parts:

- Line segment
- Word segment and
- Character segment.

Line Segmentation Process: The lines of a text block are detected by scanning the input image horizontally. Frequency of black pixels in each row is counted in order to construct the row histogram. When frequency of black pixels in a row is zero it denotes a boundary between two white pixels consecutive lines.

Word Segmentation Process: When a line has been detected, then each line is scanned vertically for word segmentation. Number of black pixels in each column is calculated to construct column histogram. When no black pixel is found in vertical scan that is considered as the space between two words. In this way we can separate the words. Figure-2.2 shows how the word is segmented.

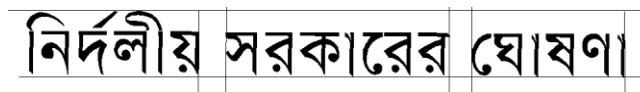


Figure 2.2: Word Segmentation

During this process, a typical situation occurs when matra less character is used in a word.

Character Segmentation Process: It is the most challenging part to build printed Bangla OCR. As Bangla is an inflectional language, the ornamentation of the characters in a word makes the segmentation difficult. A word can be constructed by:

- Only taking the basic characters like vowels, consonants and numerals
- Basic characters and consonant modifiers
- Basic characters and vowel modifiers
- Compound characters
- Compound characters and consonant modifiers
- Compound characters and vowel modifiers

To segmented character some process is follows here

Matra line detection process: To segment the individual character from the segmented word, we first need to find out the headline of the word which is called "Matra". Matra is a distinct feature in Bangla. The row with the highest frequency of black pixels is detected as matra or headline. For the larger font size, it is observed that the height or thickness of the matra increases. In order to detect the matra with its full height, the rows with those frequencies are also treated as matra [4].

Base line detection process: A baseline is an imaginary line. It is a row from where the middle zone ends and lower zone starts. Baseline and end row of the line is same when the line does not have any lower modifier(s). In a general document, it is observed that about 70% lines have lower modifiers, 30% lines do not have lower modifiers. So baseline detection is very important for Bangla. The process is described in [21]. After determining the baseline, a depth first search (DFS) easily extracts the characters below the baseline [1].

Character detection: The main parts of all the characters are placed in the middle zone. So the middle zone area is considered as the character segmentation portion. Since matra line connects the characters together to form a word, it is ignored during the character segmentation process to get them topologically disconnected [11]. A word constructed with basic characters is segmented into characters in a way by scan.



Chapter 3: Methodology

The subtasks behind building the complete OCR application are listed below:

1. Preparing Training data.
2. Preprocessing the document image.
3. Preparing Tesseract supported image.
4. Performing Recognition using the Tesseract engine.
5. Post-processing the generated text output. Among the sub tasks number 1 is independent than others. Tasks 2 to 4 are sequentially dependent on the success of the previous step.

Preparing Training Data

A complete guideline to prepare training data for Bangla script is described in [6], which is what we followed to prepare the training data. Initially, we listed 340 (50 basic, 10 vowel modifiers and 270 compound characters) characters, and considered these as the basic units for training. With these units, we performed our experiment to estimate the requisite amount of training data. In the final training data set, we consider the following combinations:

- All vowels, consonants and numerals
- Consonants and vowel modifiers
- Consonants and consonant modifiers
- Combined consonants (compound character)
- Compound characters and vowel modifiers
- Compound characters and consonant modifiers

We performed a large number of experiments during training data preparation. The experiments were necessary to find the right combination of the training data that provides highest accuracy during recognition. We prepared fourteen different sets of training data with the following parameters:

- Type of document image,
- Image DPI information,
- Font type & size,
- Segmentation, and
- Degradation.

The primary reason behind creating such large training data sets is Tesseract's limitation of supporting only 32 configuration files for each unit [5]. We automated the generation of the training images, which made the respective values of these parameters very important during the data preparation phase. The automated generation of the training data helped us to avoid the difficulties of collecting the large number of data units, such as the 3200 units described in [6], from real images. The parameter *type of document image* (TDI) encodes the source of the training data – either scanned or computer generated. *Computer generated images* (CGI) were the easiest way to generate training image from the specifications of the parameters: *image dpi* (IDPI), *font size* (FS) and *type* (FT). The intent of generating the scanned images (SI) is to make the training images more similar to the real images. To generate these images, we first prepared the paper documents following the specification of the typeface as well as the font size. Then, we scanned the paper documents with specific skew angles and also following the specification of the image resolution. We then de-skewed the images. The parameter *degradation* (DEG) was then applied to the prepared degraded images, which is applied on both CGI and SI. The application of *segmentation* (SEG) to the training images was one of the most important factors in improving the efficiency of the training data to a great extent. The reason behind applying this parameter is to make the training data units more similar to the characters/units in the test images that are provided to the recognizer. This approach of the preparing the training data significantly improves the accuracy rate. One important condition is that the segmentation algorithm must be same for both training and testing. An example of the importance of consistent segmentation is illustrated in Fig. 1 where (a) shows the training units without segmentation applied to these, (b) shows the training units after segmentation applied to these and (c) shows the condition of the test image before passing it on to the recognizer. Fig. 1 shows the significance of segmented training units, presumably because it exposes the underlying similarity between the training and testing units/characters. Also, note that applying segmentation reduces the amount of training data units. In Fig. 1(b), the unit has broken into a and o, where o is already considered as a unit to be trained and hence it is no longer necessary to train the unit . Similar condition happened for many other units among the total 3200 units, which significantly reduced the amount of training data units. An example of segmented training units of consonant ' ' is shown in Fig. 2. The top five combinations of the parameters are listed on Table 1. All of these combinations use font type of *Mitra mono*, font size of 24 and image DPI of 300. In our data preparation approach, we accumulated all character combinations in 13 images, numerals and modifier symbols in 7 images and only 1 image for degraded units.

segmentation information that we obtained from the previous stage, we generate an uncompressed 1bpp (bit per pixel) tiff image. We saved the image temporarily until the recognition output text is obtained[6].

Performing Recognition using Tesseract engine

The goal of this stage is to invoke Tesseract to recognize the temporary image and obtain the output text. There are three different ways (such as Tessnet dll file to use the API, source code and also executables) available to invoke Tesseract as a separate process to perform recognition and generate output text.

As we don't have any Bangla3.01, we have training to build our own file training data for Bengali. For this we need to have Leptonica Image processing Library and Tesseract OCR engine. And for the training purpose, we need images of Bangla text.

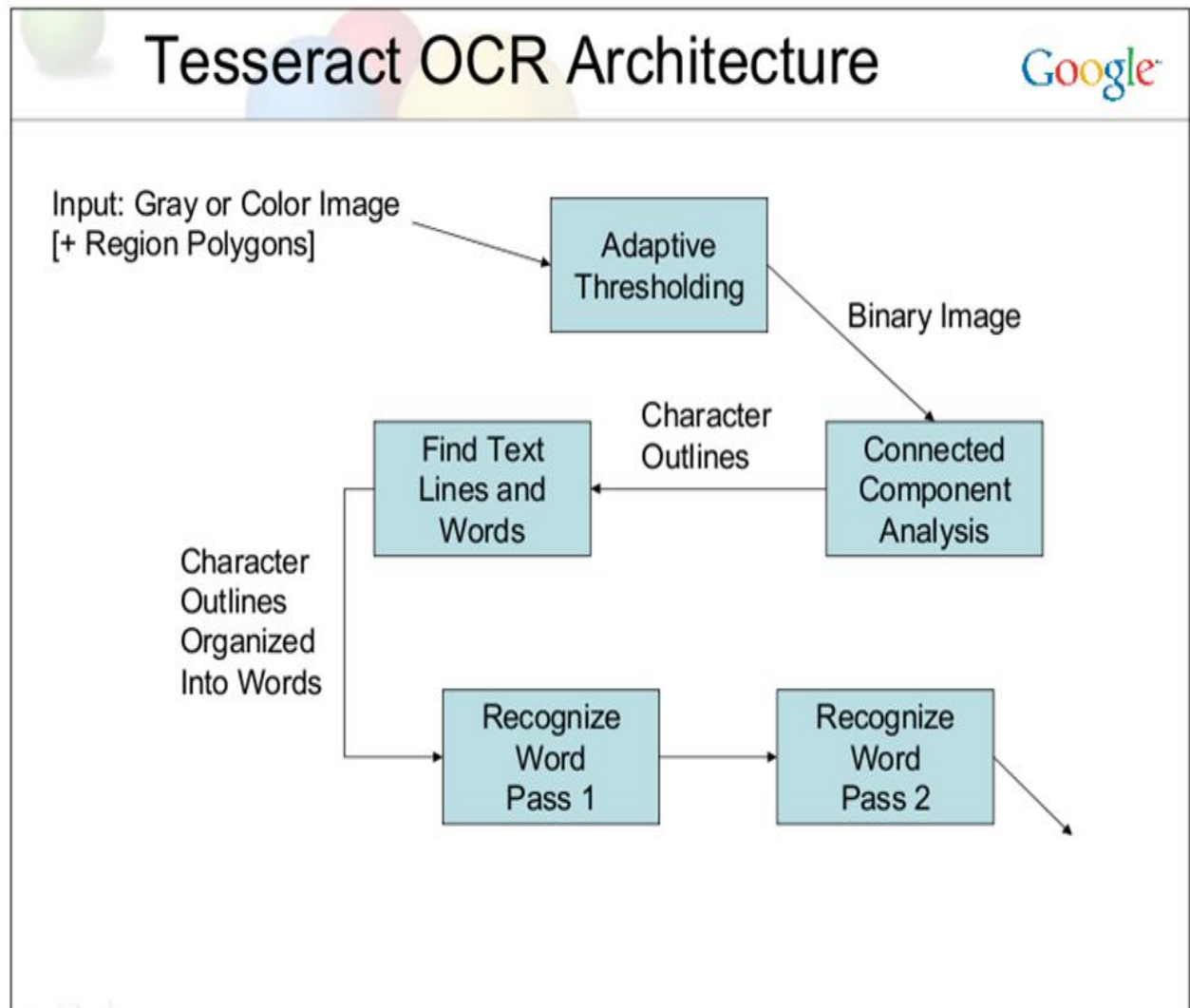
Post-processing the generated text output

In this stage we applied a two level post processing; where first level post processing is on the raw text obtained from the previous stage, and second level is applied on the first level preprocessor output to check the spelling mistakes and generate suggestions for the misspelled (out of dictionary) words.

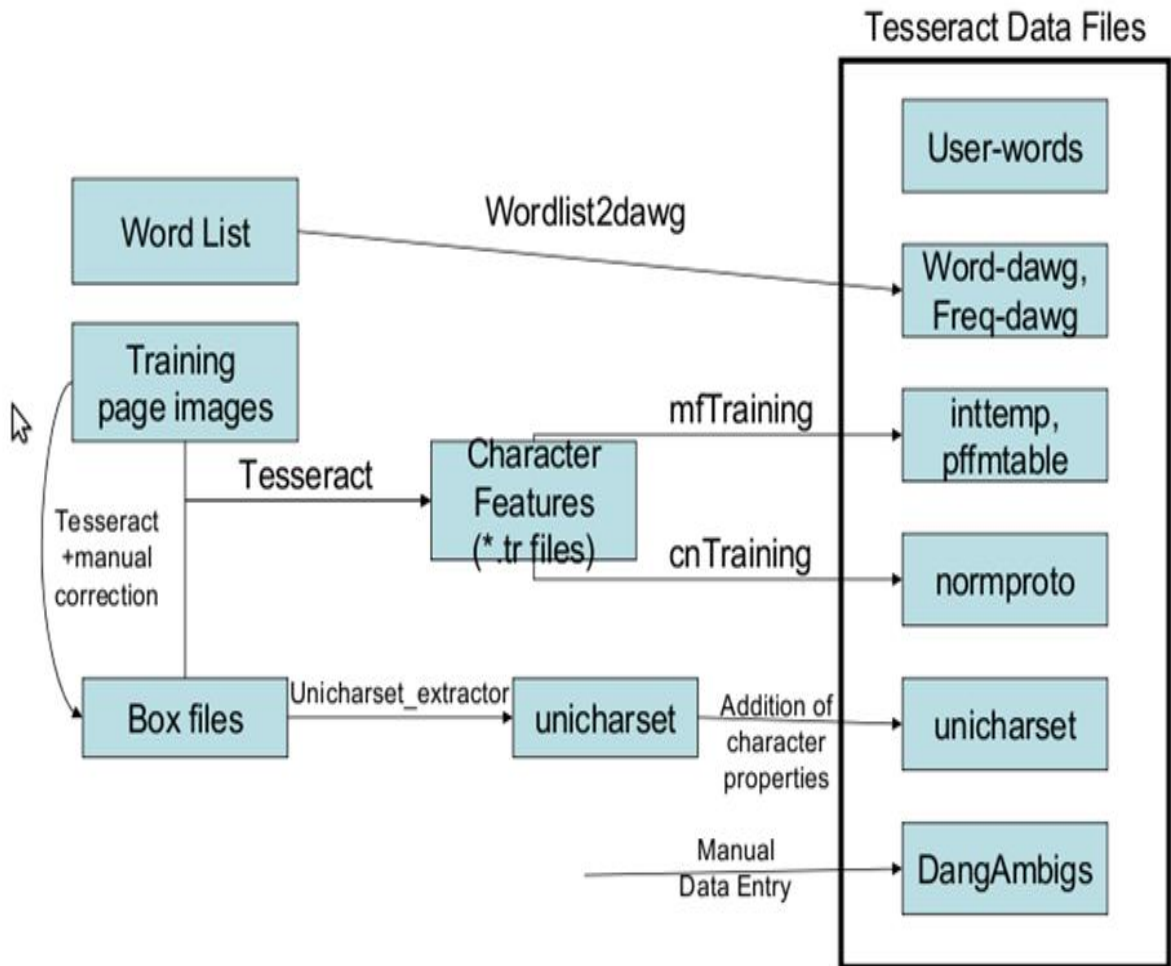
The first level post processing has two subtasks where firstly it reorder the Unicode text and then correct known recognition mistakes following specific rules. The order/sequence of characters in Bangla text image is different from the sequence of Unicode text in most of the cases where the independent vowels are present with the consonants. The independent vowels mostly suffers from this problem and hence reorder is obvious for them. In text image characters  and  appears before the consonant where in Unicode text they appear after the consonant[7].

The second level post processing invoke a dictionary consisting of 180K words. It performs spell checking operation on the text that we get from first level task. To perform the task of this level, we followed the similar post processor proposed. However, we have to find out the rules for assigning same code. Initially the code rule table is build up by assigning same code to the characters which are visually similar. Later we increase the table entry by observing the frequency of the recognition incorrectness. An example of such group of characters which are assigned similar code is listed on Table-1.1.

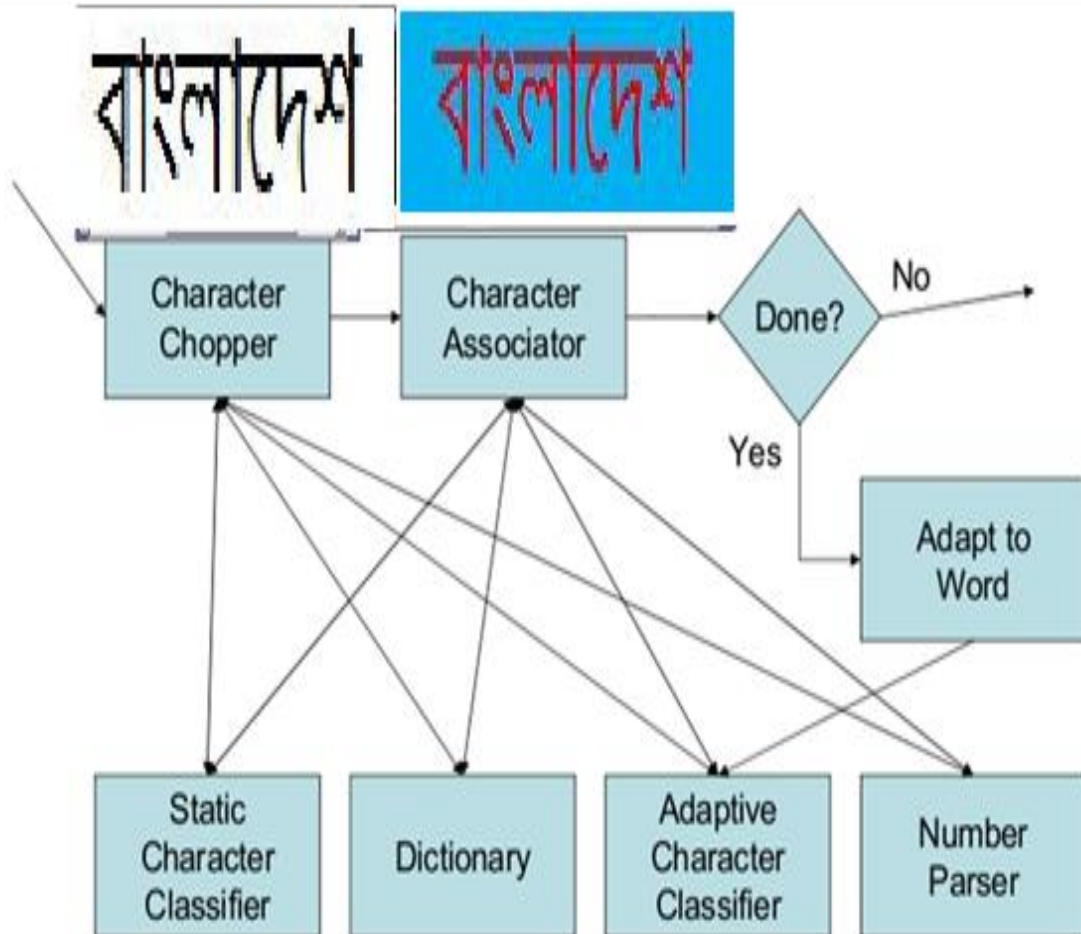
Chapter4: Working flow



Training Tesseract



Tesseract: Recognize Word



Chapter 5: Implementation

The software is developed for both Windows and Linux environment. The windows version is completely developed using Visual C++ dot Net 2010. However, to develop the Linux version we followed several steps which are listed below:

1. Integrate preprocessing algorithms within Google Open source Project OCROpus (release 0.2) as it has all necessary modules to invoke Tesseract as well as other image processing functions. Then create binary distribution of this.
2. Implement GUI in Python. The GUI has the capability to read image of any format, then invoke the OCROpus binary distribution to complete the recognition task and at last perform the post processing task using the necessary modules (written in python) integrated with it.

A complete overview of the application is shown in Fig. 3 which show the condition of the OCR application after an image (shown in the picture code box) is recognized. The left side of this GUI provides users the options to open an image, recognize an image and other preferences to enhance the image view in picture box. In the right hand side a text area will appear after recognition to show the output. At the bottom of the text area options are available to perform spell checking (2nd level post processing), save the text and close the output area.

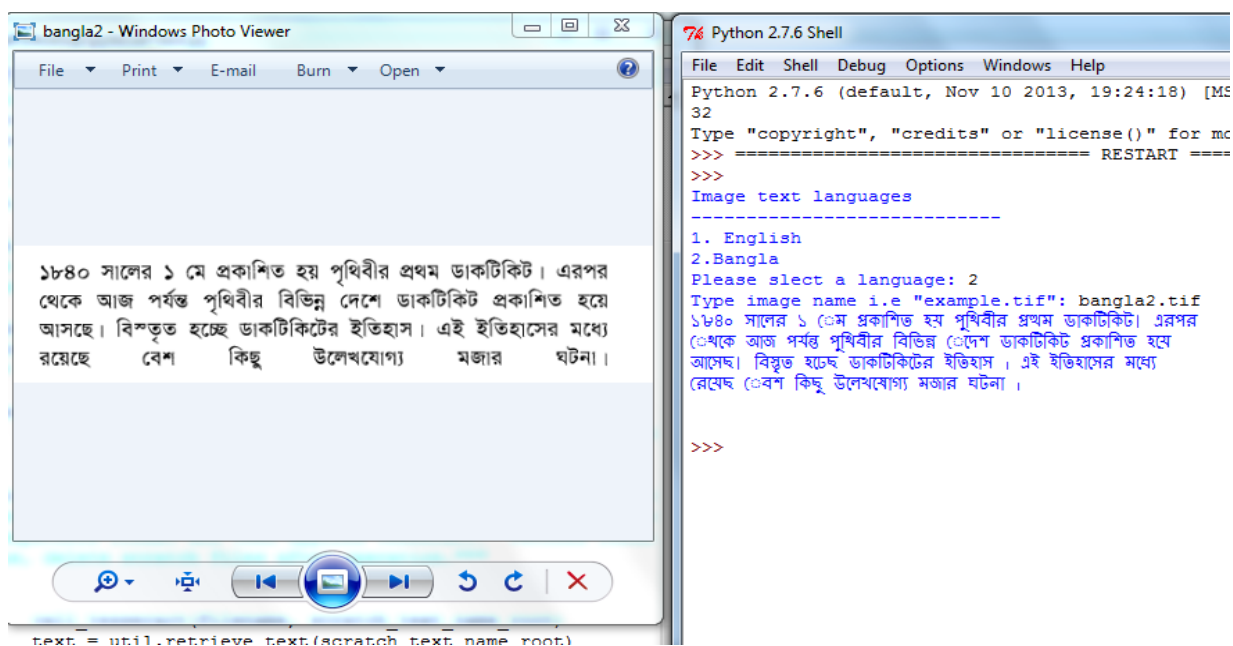


Figure 3: Complete overview of the application

The application can read image with any format. After the recognition of the image is performed, the picture box shows the temporary image which is generated from the segmentation information following the specification of Tesseract recognition engine.

Required Tools and Programming Language

This project adds the required methods on top of tesseract-tools to enable retrieving bounding boxes for words and characters recognized using OCR[8].

We successfully install tesseract using the following configuration.

- Ubuntu 12.04
- Tesseract v3.0X(3.01)
- Leptonica 1.68
- A trained data file for Bengali language.
- Python programming language (v 2.7.6)

Installation of Tesseract OCR Engine

- *extract tesseract package*
- *get inside the tesseract directory*
- *run these commands:*
 - ☐ *#./configure*
 - ☐ *#make*
 - ☐ *#make install*

Installation of Leptonica Image Processing Library

- *Extract leptonica package*
- *Get inside the leptonica directory*
- *Run these commands:*
 - ☐ *#./configure*
 - ☐ *#make*
 - ☐ *#make install*

Training Procedure for Bangla for Tesseract OCR Engine

Another advantage is that Tesseract can be trained, but unfortunately the procedure isn't well documented. The basic idea of how the engine works can be found on the figure below Figure-4. In the first three steps, the component analysis and text/word splitting has been done. Afterwards the words are splitted in characters and each character/components is passed to a 2-way recognize pass. In the first pass the results of the recognized characters/words are passed to an adaptive classifier, which uses the data as training data. After that the text will be recognized a second time but now using the adaptive classifier.

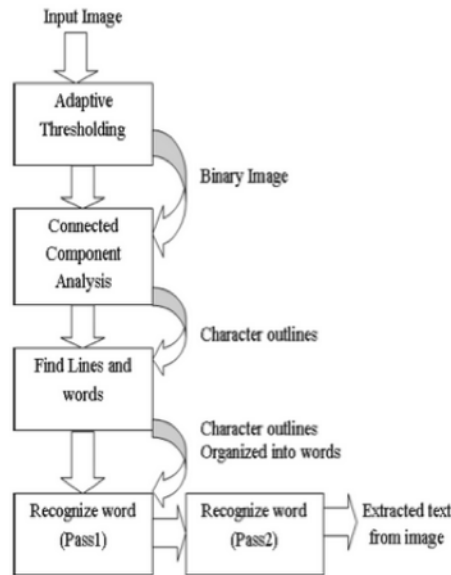


Figure-4: Tesseract training flow.

The reason why it's recognized a second time is because the first pass gives us information about the context of the text. It could be that at the end of the first pass, words are recognized which could be helpful for words in the beginning of the text. That's identically how we (most of us) humans try to learn something. The first time we will read the whole book/text to know what's exactly in the text afterwards we will read a second, third or fourth time and we try to understand everything[8].

When Tesseract is installed on a Linux system, several commands are available:

- ❖ tesseract
- ❖ unicharset_extractor
- ❖ mftraining
- ❖ cntraining
- ❖ combine_tessdata

The first command, tesseract, is to recognize images but also can be used to train and to create boxes. When we try to recognize something the result will be stored in a txt file. Below I demonstrated, tried to recognize a number (this is for my master thesis, where I used Tesseract).

0 1 2 3 4 5 6 7 8 9

```

>mahediazad$ tesseract number.jpg output
Tesseract Open Source OCR Engine V3.01 with Leptonica
>mahediazad$
  
```

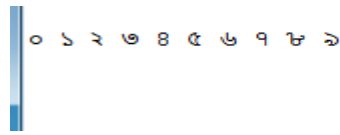


Figure-5: Output.txt

Although the recognition is almost perfect, it does recognize characters which we humans won't think are relevant. Wrong recognition can be caused by overtraining, or under-training (don't think that this word exists). With under-training I mean that an font-family is used which is very different from the originally/standard fonts. With the installation of Tesseract standard training data is delivered for the language English (on the Tesseract website there is training data available for other languages). If you would use the standard training data to recognize characters which aren't very common with the standard font-families, a low accuracy can occur[8]. That's why in some situations it could be helpful to create your own training data to increase the accuracy.

In the following steps I will explain you how to train Tesseract for your own font:

- Create boxes
- Edit boxes manually
- Extract unicharset
- Shape clustering
- Combine files
- The commands used are (I will discuss them in detail)

```
tesseract eng.matrx60x40.exp0.jpg eng.matrx60x40.exp0 batch.nochop makebox
tesseract eng.matrx60x40.exp0.jpg eng.matrx60x40.exp0.box nobatch box.train
unicharset_extractor eng.matrx60x40.exp0.box
echo "matrx60x40 1 0 0 0 0" > font_properties
mftraining -F font_properties -U unicharset -O eng.unicharset eng.matrx60x40.exp0.box.tr
cntraining eng.matrx60x40.exp0.box.tr
// rename all files created by mftraing en cntraining, add the prefix eng.
combine_tessdata eng.
```

Figure-6: Command line in Linux

Make Box

In the first step, the image we are using to train our specific font, is recognized by Tesseract and boxes are created. Each box results in a component which represents a single character. The box has an x- and y-coordinate, a width and height and the character which has been recognized in the region.

In this step we have to manually edit possible mistakes that have been detected by Tesseract. Maybe all characters were wrong because the font you are using isn't very common. To make this job a lot easier, several tools are available. These tools are called "Tesseract box editors", the tool I used is jTessBoxEditor. jTessBoxEditor is written in Java and thus platform independent, it has options to merge and split which could be handy. After you corrected the mistakes, we can create the training file. The file I used is called `eng.matrx60x40.exp0.jpg` and a box `"eng.matrx60x40.exp0.box"` will be created.

```
tesseract eng.matrx60x40.exp0.jpg eng.matrx60x40.exp0 batch.nochop makebox
```

Training file

In this procedure we will tell Tesseract the correct results. We tell Tesseract the mistakes he made so he won't make the same mistakes in a next recognition.

```
tesseract eng.matrx60x40.exp0.jpg eng.matrx60x40.exp0.box nobatch box.train
```

Unicharset

This step tries to extract the charset from the box file.

```
unicharset_extractor eng.matrx60x40.exp0.box
```

Font properties

Create a new file with the properties of the font you are trying to train. Syntax: `fontname bold fixed mitra mono`

```
echo "matrx60x40 1 0 0 0 0" > font_properties
```

Clustering

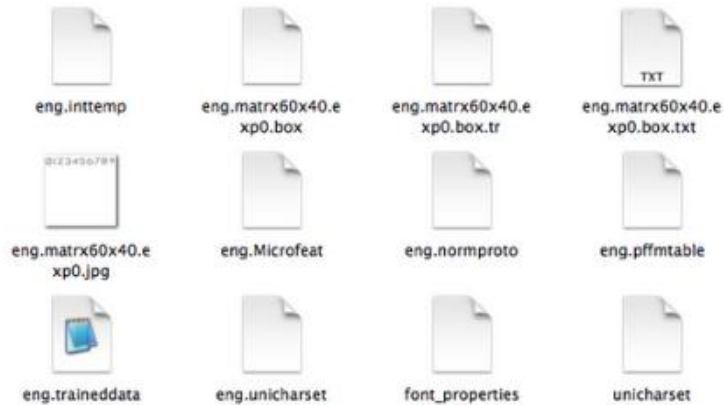
The character features are clustered.

```
mftraining -F font_properties -U unicharset -O eng.unicharset eng.matrx60x40.exp0.box.tr  
cntraining eng.matrx60x40.exp0.box.tr  
Renaming
```

We rename all the files generated by `mftraining` & `cntraining`. We add the prefix "eng." to all

Renaming

We rename all the files generated by mfttraining & cntraining. We add the prefix “eng.” As for bangle suppose to be “ban.” to all the files.



Combining

In the last step we combine all the files and the trainingdata will be created.

```
combine_tessdata eng.
```

Move trainingdata

Move the trainingdata to the tessdata folder, on ubuntu os 12.04 “/etc/opt/tessdata”.

References:

- [1] Md. Abul Hasnat, S M Murtoza Habib and Mumit Khan. "A high performance domain specific OCR for Bangla script", Int. Joint Conf. on Computer, Information, and Systems Sciences, and Engineering (CISSE), 2007.
- [2] Md. Abul Hasnat, Muttakinur Rahman Chowdhury and Mumit Khan, "Integrating Bangla script recognition support in Tesseract OCR", Proc. of the Conference on Language and Technology 2009 (CLT09), Lahore, Pakistan, 2009.
- [3] http://sourceforge.net/project/showfiles.php?group_id=158301&package_id=215908. Last accessed: April 12, 2014.
- [4] Jalal Uddin Mahmud, Mohammad Feroz Raihan and Chowdhury Mofizur Rahman, "A Complete OCR for Continuous Bengali Characters", Proceedings of the IEEE Tencon 2003, Conference on Convergent Technologies for the Asia-Pacific, Oct 15-17, 2003, PP1372-1376, Bangalore, India.
- [5] Nawsher Ahmed Noor ,BRAC University, Bangladesh "Bangle Optical Character Recognition".
- [6] Md. Abul Hasnat, Muttakinur Rahman Chowdhury and Mumit Khan, "Integrating Bangla script recognition support in Tesseract OCR", Proc. of the Conference on Language and Technology 2009 (CLT09), Lahore, Pakistan, 2009.
- [7] Saima Hossain, Nasreen Akter, Hasan Sarwar and Chowdhury Mofizur Rahman, "Development of a recognizer for Bangla text: present status and future challenges," in Character Recognition, Minoru Mori, Janeza Trdine 9, 51000 Rijeka, Croatia: Sciyo, September 2010, pp. 83 – 112
- [8] <http://blog.cedric.ws/how-to-train-tesseract-301>. Last accessed: March 10, 2014.
- [9] <http://www.resolveradiologic.com/blog/2013/01/15/training-tesseract/>. Last accessed: April 12, 2014.